Concealing Secrets in Plain Sight: An Attempt at Steganography with Language Models in News Articles

Kim Jana Eiken (402003), Bautista Galmarini (500443), Hristo Boyadzhiev (390932)

July 19, 2024

Abstract

The public's interest in LLM-based applications has grown steadily as a result of the large language models' (LLMs) rapid advancements. But as LLMs become more accessible, the blurring of the lines between human-written and machine-generated text introduces new risks, such as disinformation campaigns, social engineering, and electoral tampering. In order to overcome these difficulties, watermarking techniques are being studied, which discreetly encode patterns in machine-generated text to enable origin recognition. This paper explores the use of LLaMA models in conjunction with watermarking techniques to conceal confidential data in text documents, using news articles as a dependable and comprehensible text source. The aim is to discreetly hide information in the human-generated text that is recognizable by an algorithm-aware co-conspirator, but to the perspective of anyone else is unrecognizable from unaltered human text. We introduce approaches both to hiding and restoring secrets as well as for detecting when secrets are hidden. This report presents the results we had from implementing those approaches. It covers their theoretical foundations, implementations and drawbacks. Overall, the study aims to contribute to the ongoing research on enhancing the security and reliability of text data in the era of large language models.

1 Introduction

In recent years, the field of large language models (LLMs) has witnessed significant advancements, driven collaboratively by both academic and industrial efforts. Notably, the introduction of ChatGPT[1], a powerful AI chatbot build upon the GBT models, was as a remarkable milestone that has earned considerable attention [ZZL+23]. LLMs, characterized by being trained on massive text datasets, exhibit strong human-like capabilities in understanding natural language and excelling at complex tasks, e.g. writing documents or executable code, from textual instructions or from a few examples [KGW+23a]. Amongst OpenAI's GBT models, many language models have been presented including LLaMA [TLI+23]. Based on the assumption suggested in [HBM+22] that the amount of data a model is trained on is pivotal for its performance, rather than the size of the model itself, Meta developed the LLaMA series of language models for various inference budgets. The models are trained exclusively on publicly available data and were released to be used by the community.

With the rise of LLMs, new challenges and risks are emerging, particularly as the distinction between machine-generated and human-written text becomes increasingly difficult. These risks include social engineering, election manipulation, fake product reviews, creation of fake news and web content, as well as cheating in academic settings [MDK+23, BAS+22, KGW+23b, KGW+23a]. To overcome these challenges new research suggest the use of watermarking approaches [KGW+23b, KGW+23a, HXZ+22]. The idea is to integrate subtle patterns into machine-generated text, such that the origin of the text can be detected [KGW+23b]. These watermarks should not be noticeable without having knowledge about the algorithm that has been used to create them.

Watermarking is part of the field of steganography, the process of encrypting data within another medium so that its presence is not evident to the human eye [EMM20]. While the main aim for the usage of watermarking approaches is to encrypt information about the author, this inspired the the idea to use watermarking techniques to hide other information. The embedding and therefore protection of information in digital objects allows them to be safely transferred through open communication channels. In this report we explore how LLM's can be used to conceal secret information in documents. In particular, we will use one of LLaMA's models to modify textual input. We present the use of highlighted indices inside text documents corresponding to

[1] https://openai.com/blog/chatgpt/

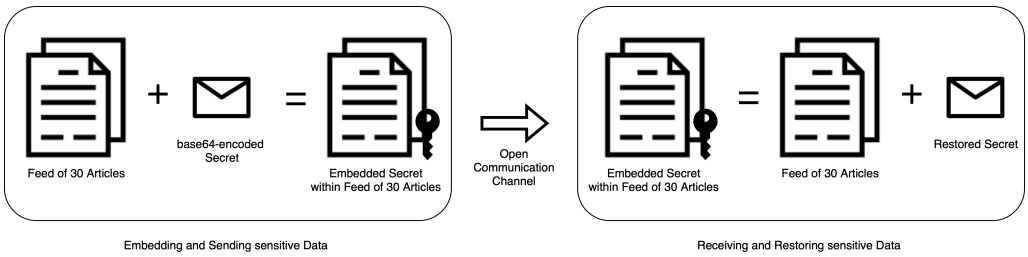

Figure 1: Basic concept of a hiding and restoring a secret embedded into a collection of news articles which are transmitted openly.

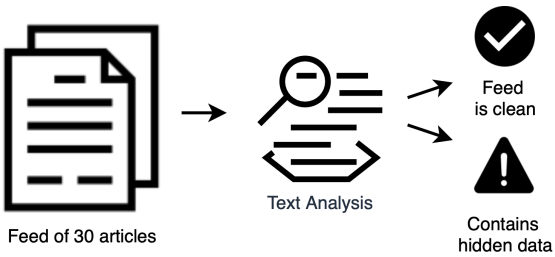

Figure 2: Basic concept of a detector deciding wether a feed of news articles contains any (unidentified) information.

byte-encoded information for hiding secrets. The target medium in which we will embbed the information are news articles, since they offer a reliable stream of openly accesible text documents with hundreds of artcicles published every day. As part of the hiding process we will present an algorithm to retrieve the embedded information from the articles by discovering parts of the article that stand out and decoding the embedded information.

To furthermore discuss the degree of security that text-based steganography approaches offer, we developed a script to detect possible occurances of concealed data within articles. While the focus of steganography is to hide information such that it is not noticable by humans, in this technoglogy-driven age it is important to consider tools, e,g, for text analysis, when dealing with sensitive data.

Problem description

This report is done in the scope of a university project course called "Steganography with Language Models" at TU Berlin. The task of this project course was to hide a piece of secret information ("secret") in a feed of articles. Specifically, the secret is a base64-encoded sequence 32 bytes in length. In each feed there were 30 articles of length roughly between 500 characters and 500 words.

Each team submitted scripts for a "hider", "restorer", and "detector". The hider takes an example, unseen input article feed along with an unseen secret and produces the same feed with potentially altered articles. The same team's restorer takes as input their hider's output for this example and must extract ("restore") the 32-byte secret. The restorer is graded on the accuracy with which the secret is recovered.

The detector gets as input only a feed of articles and must determine whether they are unaltered, human-generated texts ot there is a secret hidden inside. To test detectors, each team's hider output is computed for an example feed and secret. These outputs are then mixed with unaltered article feeds and fed to the detector, through which precision and recall can be computed.

2 Hider and Restorer

While we agreed on our main approach (Byte indices approach) early on, we started this project by implementing a simple hider (Simple embedded link approach). The intention behind this simple hider was to play around with LLaMA and get a feel on how to efficiently make use of LLaMA to hide a base64 string into a feed of 30 articles. During the project we then implemented our main approach and tried to optimize it, resulting in the third approach (Context-fitting byte indices approach). In the following we are going to present our three approaches along with some implementation details and drawbacks for each.

2.1 Simple embedded link

The first hider we submitted placed the base64-encoded secret in its original form into an artificially constructed web link. Our detector would then simply find the web link using the format with which we constructed the link. In the early parts of the competition, this approach performed well. Trivially, the restorer works perfectly.

While playing around with LLaMA, among other things, we let LLaMA generate new tokens to extend the given articles by giving LLaMA the article as prompt input. We noticed that LLaMA included links containing some random id (e.g. "Xy4") in the extended articles, meaning the occurrence of some "Xy4" is, according to LLaMA, not unlikely. Out of curiosity we checked the probabilities for the base64 secret embedded in the link, replacing the "Xy4". The result, as expected, were high probabilities (low logprobs) for "random" characters following other "random" tokens, with an average logprob of -5.57 (Min: -13.07, Max: -2.90).

One possible reason for why the hider initially worked well was that it did not alter the human-written text itself, so it would not be trivially detectable by an LLM looking at the text, unless the LLM was specifically instructed to look for links or base64 strings. More fit to detect this hiding strategy would be a non-LLM test for certain patterns, such as links or base64 strings (more on that in 3). Later in the semester, once more of the opposing teams implemented such checks, their detectors were able to consistently classify the text as altered.

2.2 Byte Indices

Inspired by [KGW+23a], we came up with the idea of encoding the secret by changing the output word slightly (e.g replacing it by a synonym), using the LLM's output. The restorer would then have to find those indices at which the chosen word is "uncharacteristically" unlikely.

Concept

Tokens Unidirectional LLMs like LLaMA are trained to predict the next piece of text based on the preceding ones. Since in non-hieroglyphic languages individual characters hold little to no meaning, succeeding to encode semantic information character by character would require computationally prohibitively wide embeddings. Many of the world's most widely spoken languages build words out of morphemes, which, as opposed to characters, do hold semantic information. During training, the LLM uses this by training a tokenization model (or "tokenizer"), intended to split words into semantically meaningful parts, referred to as "tokens". This tokenizer is then fixed, defines the model's "dictionary" of tokens and can be used to tokenize any text in a way that the underlying model would be able to predict on it.

LLM output A language model is then trained to predict the next token based on a sequence of tokens preceding. Common LLMs' final layers have a softmax activation [VSP+23], which normalizes the output to a probability distribution over all tokens in the model's dictionary. Importantly, given a seed, the model LLaMA we used is deterministic. This means that given the same context, it will always output the same distribution, a characteristic useful for restoring. The distribution is represented as log-probabilities, which we will henceforth refer to as "logprobs".

Splitting the secret Base64 is a type of string encoding for any binary data. In order to reduce the amount of information we must encode at any part of the text, we can split it into sequences of any number n of bits, e.g. into bytes. Each n-bit-sequence can then be interpreted as an unsigned number ranging from 0 to $2^n - 1$

Secret as indices For small enough n so that $2^n - 1$ is smaller than a typical article length, these unsigned numbers can be interpreted as indices. An approach to hiding and restoring is hiding in the tokens at those indices any characteristic that can be recovered by a restorer that is aware of what to look for.

Defining the rule to flag an index as hiding a secret proved challenging. Across indices of a text, both the highest predicted logprob and the difference between it and the logprob of the human-generated token used in the original text vary wildly (see Figures 3 and 4), making it impossible to set a global logprob boundary by which to decide if the token used in a text is "uncharacteristically" unlikely.

In order to alleviate that we went instead for tokens with significantly lower probabilities than the original token at the index. Since the model we used predicts unidirectionally, that is, only

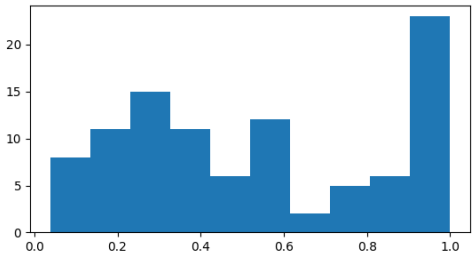

Figure 3: Histogram of probabilities of the most likely token across an article

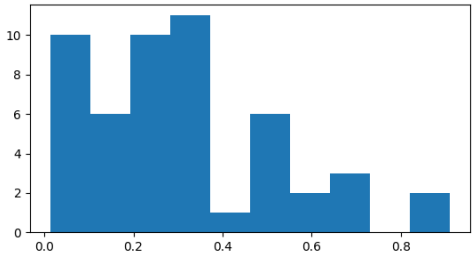

Figure 4: Histogram of the differences between the probabilities of the most likely and the actual token across an article

looks at tokens that come before, the output tokens would usually not fit the original human text that came after. For this reason, after the first token is generated for an article, the text that comes afterward is generated using the LLM as well.

Implementation

Given a feed of articles and a base64-encoded secret, the hider produces an altered feed encoding the secret. First, the secret is decoded into a list bytes allowing us to treat the secret as a list of numbers. The hider iterates through the feed entries, hiding N_SECRETS bytes per entry. We fixed N_SECRETS=3, since we are working with a numbers between 0-255 and feed of 30 articles around 500 words each. The hider uses a LLaMA model (LLaMA-2-7B-GGUF) to tokenize the articles, calculate token choices and their probabilities and to extend articles by generating text. It is therefore necessary to use the same model for restoring the secret.

The algorithm hides a set of bytes {a, b, c} in an article by going to the a-th token in the article, generating the token's probability distribution for that position and choosing a token from the distribution that is not the most probable one. We spend a lot of time testing different variations with the example feed and ended up generating the top 100 tokens and choosing the token with the lowest probability. This results in a high computation time, however generating fewer choices was often times not significant enough and the bytes were hence unrestorable. Ideally, we would have tested if the 100th choice is significant enough with a larger dataset. An alternative would be to use a random word generator for a given context and only checking if the probability is low enough. The a-th token is then replaced by the chosen token. Using the model, the article is extended until there's at least b more tokens, replacing the a+b-th token. These steps are then repeated for c, replacing the token at the position of the cumulative sum of the three bytes. If no secret bytes are left to be hidden, the remaining articles are extended with the model to the average length of the altered articles to blend seamlessly.

Token Position	Token Choice	logprob
241	' cover'	-7.641354168821565
440	' believed'	-6.967935066638399
559	'F'	-17.106016370925854

Table 1: Exemplary token choices of the 100th most probable token for the token position according to LLaMA and their logprobs.

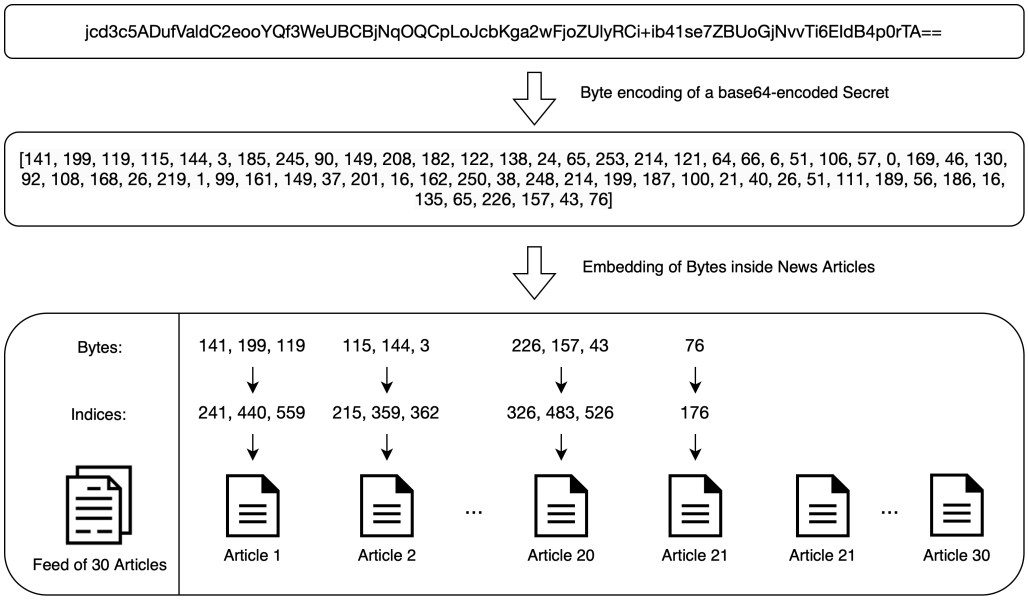

Figure 5: Exemplary embedding of a base64-encoded secret into a feed of 30 articles with our byte-indices hider. See Figure 6 for exemplary encoding of article 1.

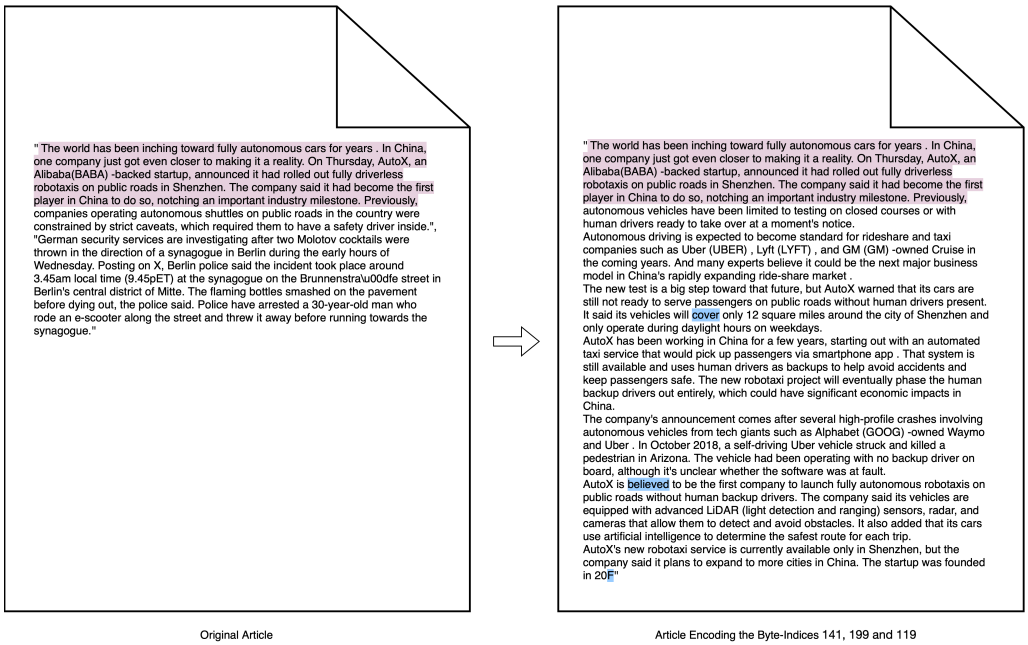

Figure 6: Exemplary output of our byte-indices hider ignoring the first 100 tokens (highlighted pink) and embedding three bytes of the secret by choosing low propability tokens (highlighted blue) at the corresponding token positions. See Table 1 for the token propabilities. Non-highlighted text was generated with LLaMA.

Limitations

A problem we faced while extending with LLaMA, is the refusal of the model to generate more tokens at given points. While we can define the maximum amount of tokens to be added, there is no parameter for the minimum tokens. We had to check each time if we have enough tokens to hide the next byte, possibly trying again and again to extend the article. Fortunately, re-extending was not necessary the majority of the time. We noticed one article in particular of the test feed. However, since the hider should work on all kind of feed, simply ignoring the article was no option. We tried to explicitly add a word like "Also" in hopes of stimulating the model. While we in fact received more tokens from LLaMA, the added word destroyed the probability distribution necessary for restoration. Therefore we just added a loop, increasing the requested maximum amount of targets. On our test feed we eventually got a long enough article, but this is a risk for a possible endless loop.

The biggest problem with this hider was to restore the secret. To optimize the restoration of the started each article with an offset of 100 tokens, in which no secret bytes are hidden. We noticed that the restorer almost always identified a secret token wrongfully in the beginning of an article, since the first tokens are mostly not probable according to LLaMA. As another step to better restore the secret, we cut the article at the 100th token and only generated text after. The generated text has a more homogeneous probability distribution than the given articles. While this let our restorer to correctly find the secret on our small test dataset, we assume that this is the reason why most detectors classified the hider output as suspicious. This also increased the computation time, having to use LLaMA around six times per article.

2.3 Context-fitting Byte Indices

While unfitting tokens can be camouflaged by generating the segments after the token, due to the intentionally lower probability the inserted token itself might still not fit the context that precedes it. Therefore we attempted an approach of selecting the first token with a probability lower than the original token that "fits the context" instead of the lowest probable token for that position.

Some LLMs like BERT [DCLT19] are "masked", meaning they can predict a token using context both before and after it. The most modern models however, including the one used in this project, are unidirectional.

However, these newer models have context windows so long that they can contain the entire article, including any context that comes after the word. In order to use this, at each "relevant" index (in the same sense as in the Byte Indices approach) we used the LLM twice: once to generate its conventional unidirectional output, and once again with the prompt scheme

`Which of the following tokens would you pick to fill the blank marked [[BLANK]] in the sentence <<MASKED SENTENCE>> \n <<TOKENS>>`

where `<<MASKED SENTENCE>>` is the sentence in which we are replacing a token with the token on the relevant index replaced by `[[BLANK]]` and `<<TOKENS>>` is collection of the tokens in the unidirectional output which have a **lower** logprob than the original human token, ordered by logprob, highest to lowest. We were able to prove the concept by successfully replacing a word by its synonym within one of the example articles.

The restorer direction would then look in a similar way: for each index we first generate the token distribution and take from it only those tokens which have a **higher** logprob than the one used in the text. Then we supply those words again ordered by logprob from highest to lowest. The prompt would then ask for the first word in the list that fits the context of the sentence.

One drawback of this apprach is that the restorer might see indices as hiding a secret simply because there exists a token higher in logprob that also fits the context well, which is not impossible for human-generated text. Additionally, the LLM has to be prompted twice at each index of each article, a computationally expensive endeavor.

3 Detector

Our plan was to develop a LLM-based detector, analyzing the likelihood of each token per article. However our other detector, which we developed simultaneously, gave us pretty good scores. Only towards the end, with optimization of the competing hiders and detectors as well as with improved Feedback, did we notice losses with our detector. Our detector surprisingly misclassified clean feeds.

3.1 LLM-based Approach

This idea relies on the assumption (A1) that the used LLM, in this case LLaMA, classifies the tokens of the actual, untampered articles as somewhat likely. As well as the assumption (A2) that hiders fed with unknown articles will produce articles containing tokens categorized with low probability by the LLM. We discovered relatively early on with the provided feed of articles and our "simple hider" that A1 and A2 do not hold per se. We found probability ranges from 0 to -15 in the example feed, which does not differ much from the range of the plain embedded base64 secret by our simple hider. Therefore categorizing each token with a logprob below a certain threshold is not sufficient. With the hypothesis (A3) that tampered articles contain tokens at positions for which tokens with a significantly higher probability exist, we adapted the detector to compare tokens to the best choice according to the LLM for the position. The detector hands each article to the model without generating new text, to calculate two token choices per token position. The first of the two token choices represents the most probable token for that position and the second on represents the given token or the second most probable token if the given token is the most likely one. We can then check if the gap of the two logprobs is too high, while we limit the comparison to tokens with a probability below a certain threshold. While we hoped this approach would give good results in general, it is inspired by our hider approach of choosing a token with low probability in relation to the position. However, even the gab is not measurable by threshold, i.e. a high gab is not a good indicator for a hidden secret. The range of the different logprobs per token position differ extremely based on the token, i.e. the maximum logprob for some token position may be a low or medium logprob for another token position. The detector has a high rate of false positives. Another downside of the LLM-based detector is the high computation time. We did further work and testing for the LLM-based detector, for example checking if the logprop of the tokens used in the articles are equal or less than the xth-choice for that token position, similarly to our restorer. However, we noticed better results with text analysis detector approach and therefore focused our time on optimizing the other detector.

3.2 Text Analysis Approach

The developed stenographic detector presents a comprehensive and robust approach to identify concealed information in texts. Below is a detailed description of the functionalities and the main approach of the detector:

3.2.1 Reading and Validation of JSON format

The process begins with reading the JSON format, ensuring that the data structure is consistent and complies with the established requirements. Validation ensures the integrity of the input data, providing a solid foundation for subsequent analysis. With this, we can easily "detect" that some information has been manipulated if other groups were not able to fulfill the requirements.

3.2.2 UTF-8 Encoding Check

Each "feed" undergoes a thorough UTF-8 encoding check. If any discrepancies or content incompatible with UTF-8 are detected, the detector returns a True value. This functionality is crucial to ensure the correct interpretation and manipulation of the text during subsequent analysis stages. This does not allow other groups to use invisible or strange characters which are usually not visible to the human eye.

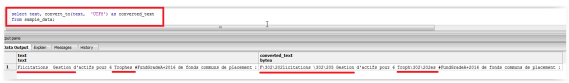

Figure 7: Example of hidden information in non-UTF-8 Encoding.

3.2.3 Text Preprocessing

Before initiating steganography analysis, the text undergoes preprocessing. This includes the removal of special characters, normalization of uppercase and lowercase, and other techniques aimed at optimizing consistency and the effectiveness of subsequent analysis.

3.2.4 TF-IDF Analysis and Anomaly Detection

Term Frequency-Inverse Document Frequency (TF-IDF) is a widely used technique in natural language processing and information retrieval. It quantifies the importance of each term within a document relative to its occurrence in a larger collection of documents. In the context of the text hidden message detector, TF-IDF is applied to the preprocessed text to create a numerical representation that captures the significance of each term. This representation reflects the unique characteristics of the document compared to a broader corpus.

An Isolation Forest is a machine learning algorithm designed for the detection of anomalies or outliers in a dataset. In the detector, the TF-IDF representation of the text is subjected to the Isolation Forest model. The model operates by randomly selecting features and isolating instances until anomalies are identified. Anomalies are instances that require fewer splits to be isolated, indicating they are different from the majority of the data.

The Isolation Forest excels at identifying instances that deviate from the norm, making it effective in detecting anomalies introduced by hidden messages. Unusual patterns in the distribution of terms and their frequencies, indicative of potential hidden information, are flagged as anomalies during the analysis.

The results from the TF-IDF analysis and Isolation Forest are distilled into a scoring mechanism. Higher scores indicate a higher likelihood of the presence of hidden messages. Decision-making involves setting a threshold score, beyond which a "feed" is classified as containing a hidden message. This careful calibration helps in balancing sensitivity and specificity, minimizing false positives and negatives.

The TF-IDF analysis and anomaly detection [Che21] component operates synergistically with other elements of the detector, such as the UTF-8 encoding check and the search for encoding patterns. The combined results contribute to the comprehensive decision-making process, ensuring a robust identification of hidden messages in text.[2]

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$
$$IDF(t) = log\frac{N}{1+df}$$
$$TF-IDF(t,d) = TF(t,d) * IDF(t)$$

Figure 8: TFIDF Formulas where d refers to a document, N is the total number of documents, df is the number of documents with term t. TF-IDF are word frequency scores

In summary, the TF-IDF analysis coupled with the Isolation Forest algorithm serves as a crucial component of the detector, allowing for the nuanced identification of anomalies in the distribution of terms and frequencies, indicative of potential hidden information. This method enhances the overall effectiveness of the text hidden message detector in diverse scenarios.

3.2.5 Search for Encoding Patterns

The detector addresses the diversity of steganography techniques by searching for specific encoding patterns. Various methods are examined, including Base64, Binary, Morse, Hexadecimal, as well as the detection of links and white spaces. If any of these methods return a True value, the "feed" is marked as containing hidden information.

The detector provides a consolidated result based on the combination of individual results from each stage. If any step returns a True value, it is concluded that the "feed" contains hidden information. This careful and stratified decision-making allows for precise detection and minimizes false positives.

The main approach of the detector incorporates both traditional techniques and advanced methodologies, offering a comprehensive solution for identifying hidden messages in text. The combination of TF-IDF analysis, encoding pattern detection, and UTF-8 encoding validation provides a versatile and effective tool in the fight against steganography in JSON format documents.

[2]https://kinder-chen.medium.com/introduction-to-natural-language-processing-tf-idf-1507e907c19

3.3 Exploration of Alternative Approaches and Reasons for Non-Inclusion

Throughout our research journey, we diligently explored various alternative methodologies to enhance the effectiveness of our hidden message detection system. However, after meticulous consideration, certain approaches were deemed impractical for integration into the final model. The decision to exclude these methodologies was driven by several key considerations.

3.3.1 N-grams Analysis (Sequential Word Sequences)

N-gram analysis is a fundamental technique in natural language processing (NLP) and computational linguistics that enables the extraction of valuable insights from sequential data, such as text. An N-gram refers to a contiguous sequence of 'n' items (or tokens) extracted from a given text, where 'n' represents the number of elements in the sequence. This analytical approach plays a pivotal role in uncovering patterns, relationships, and contextual information within a body of text.

The initial step involves breaking down the text into individual tokens, which can be words, characters, or even phrases, depending on the granularity desired for analysis. Once tokenized, the text is processed to create N-grams by forming sequential combinations of 'n' tokens. For instance, in a bigram (2-gram) analysis, every two consecutive tokens form a distinct N-gram. N-grams are then counted to establish a frequency distribution. This reveals how often specific sequences of tokens appear in the text, shedding light on recurring patterns. Beyond basic frequency counts, statistical measures, such as probabilities or information entropy, can be applied to discern the significance of certain N-grams. This aids in identifying patterns that might be indicative of specific linguistic structures or recurring themes.

As 'n' increases, the number of possible N-grams grows exponentially. This can lead to sparsity issues, especially with smaller datasets, and may require techniques such as smoothing to handle unseen N-grams. N-gram analysis relies on local context and may not capture long-range dependencies in language. Techniques like neural language models are designed to address these contextual limitations. [Aga22]

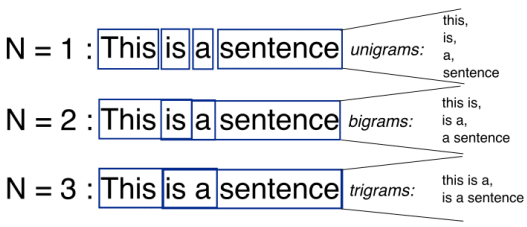

Figure 9: Unigram, Bi-gram, and Tri-gram Model

The implementation of N-grams analysis introduced a substantial computational overhead, jeopardizing the overall efficiency of the detection system. The outcomes from this approach did not align with our anticipated results and failed to offer relevance within the specific context of detecting concealed messages in text.

3.3.2 Entropy Analysis, a Measure of Randomness

Entropy analysis, rooted in information theory, is a powerful mathematical concept employed in various fields, including information theory, physics, and data science. In the context of natural language processing and data analysis, entropy serves as a measure of uncertainty or disorder within a system. Entropy analysis is particularly insightful when applied to diverse datasets, offering a quantitative lens through which the unpredictability and information content of the data can be assessed.

A higher entropy value indicates greater disorder or unpredictability within the dataset. In contrast, lower entropy suggests a more ordered or predictable dataset. For instance, a dataset with all identical elements would have minimal entropy, as each element's probability of occurrence is certain.

Entropy analysis is utilized in cybersecurity to detect anomalies and assess the randomness of data patterns. Deviations from expected entropy levels can indicate potential security threats. We

thought that this type of analysis was going to be extremely good but the effectiveness of entropy analysis depends on the context and the underlying assumptions made about the data.

We have to add that entropy analysis can be sensitive to outliers or extreme values within a dataset, potentially skewing the assessment of disorder. The application of entropy analysis, while theoretically promising, imposed considerable demands on computational resources. This manifested as a challenge, casting shadows on the practical feasibility of integrating this methodology into our tool. Additionally, the insights gleaned from entropy analysis failed to exhibit pertinence to our specific scenario of hidden message detection in textual content.

3.3.3 TF-IDF Implementation for Char Analysis

As we have mentioned before, TF-IDF is a powerful technique commonly used in natural language processing for extracting meaningful information from text data. However, when applied to the character level, specifically for character analysis, certain pitfalls emerge that may undermine the effectiveness of the analysis.

At the character level, text data is inherently noisy, encompassing a wide range of symbols, punctuation, and special characters. TF-IDF, designed for word-level analysis, struggles to discern meaningful patterns amid this character-level noise. As a result, the analysis may become inundated with irrelevant information, hindering the precision of hidden message detection.

Unlike words, individual characters lack the semantic meaning inherent in language. TF-IDF, which relies on the significance of terms within a document collection, may inadvertently prioritize characters with high frequency but limited semantic value. This can lead to misinterpretation of the true nature of the hidden message.

Performing TF-IDF analysis at the character level substantially increases computational complexity. The sheer volume of characters within a document can lead to longer processing times and resource-intensive computations, making it impractical for real-time applications or large datasets.

TF-IDF character analysis operates on a local level, focusing on the frequency of individual characters without considering their context within words or phrases. This lack of contextual understanding diminishes the ability to capture the nuanced patterns and structures indicative of hidden messages in text.

In conclusion, the application of TF-IDF at the character level was considered but ultimately discarded due to the complexities and limitations it introduced, especially in the context of detecting hidden messages. Recognizing these challenges, alternative approaches that leverage higher-level linguistic features and advanced methodologies emerged as more viable and robust solutions for uncovering concealed information in text.

3.3.4 Low Probability Token Analysis

The concept of analyzing low probability tokens in comparison to other options for that token involves assessing the likelihood of a particular token occurring in a text relative to alternative possibilities. While this approach may seem theoretically insightful, its application in our specific context, particularly in hidden message detection, introduces several challenges and renders it a less-than-ideal choice for inclusion.

Implementing an analysis that involves comparing the probability of a token against various alternatives can result in a computationally intensive process. This heightened computational demand poses a significant drawback, especially when aiming for real-time application or processing large volumes of text.

Tokens, especially in the context of hidden messages, may not always carry clear semantic significance when considered in isolation. Focusing on low probability tokens without a comprehensive understanding of their contextual relevance may lead to misinterpretation and an increase in false positives.

The computation of probabilities for each token against alternative options requires intricate statistical analyses. This complexity not only adds to the computational burden but also introduces challenges in determining accurate probabilities, particularly in the absence of extensive training data.

Establishing a threshold for what constitutes a "low probability" token can be a subjective task. The lack of standardized criteria may introduce variability in results and hinder the reproducibility of the analysis across different datasets or scenarios.

In the context of a practical and real-world application, the computational complexity, lack of semantic clarity, and sensitivity to encoding variations make the low probability token analysis less

viable. The goal of an effective hidden message detection system is to balance accuracy, efficiency, and adaptability.

Considering these factors, the investigation into analyzing low probability tokens in comparison to other options for that token underwent a thorough evaluation and was excluded from consideration. The inherent complexities associated with this approach, along with its constrained adaptability to the intricacies of hidden message detection, prompted us to prioritize alternative methodologies that could more effectively align with the specific demands of our research context.

4 Conclusion

During the project we were able to implement a few steganography approaches using LLM's to conceal secret information in a collection of news articles. We developed a simple hider, inserting the secret into a web link, which was surprisingly hard to detect using LLM's. However, this approach would definitely not be suitable for real world application, since it is easily detectable with text analysis tools and also by the human reader.

We spend most of our time developing and optimizing a byte-indices approach, converting a secret into bytes, treating them as token positions from 0-255 and highlighting those positions by inserting low probability tokens. We used the LLaMA model LLaMA-2-7B-GGUF to tokenize the articles and generate new tokens for different positions, comparable by their probability scores. Our hider cuts the original article after the 100th token and generates new text using the LLM, while embedding the secret. At the secret token positions the 100th likeliest token is inserted into the article. We encountered a few difficulties with the restoring part for this approach. Tokens in human-written texts vary extremely in probability, making it hard to differentiate between "normal" tokens and highlighted once. Especially the first few tokens of any text exhibit low logprob scores. The probability scores for different token positions aren't comparable. While some positions have probability ranges from -10 to -15, others have 100 token options with probabilities higher than -1. This makes it impossible to use some sort of a threshold to determine abnormal token choices. In the end, we decided to generate a certain amount of token choices per position, comparing the probability of the actual token used in the article to the one of lowest token choice for each position. We declared the token as abnormal if the probability scores are equal, e.g. there are at least 39 more probable tokens for that position than the one found in the article. We had to limit the generated token choices per position to 40 because of the computational complexity. While we succeeded in restoring the secret for an example feeds, we worry that some bytes can not be restored based on the discrepancy between the 100 generated choices in the hiding part compared to the 40 in the restorer. We experimented with many different settings, however embedding a token choice less than 100 lead to many undetectable indices.

In order to reduce the amount of contextual unfitting choices, we attempted to optimize our hider. Although we did not get a better approach fully working, we were able to use an unidirectional model to generate bidirectionally. Apart from the problems with retrieving the secret and the high computational complexity, our hider was not able to deceive the best detectors of the competing teams. While this approach should in theory work well at concealing information from the human reader, the text between our embedded tokens generated with LLaMA was probably to homogeneous. Together with the probability spikes and contextual unfitting tokens throughout the articles, this approach would need more research to be used for real world examples.

Apart from developing steganographic approaches for hiding and retrieving data, we explored different ways of detecting if additional data has been hidden within a document. Through the use of methods that ranged from TF-IDF analysis to UTF-8 encoding checks, our goal was to develop a detector that could accurately determine if information was hidden in text documents. But this project also brought to light the shortcomings of several approaches in the particular setting of textual concealed message detection.

To sum up, this project has given us important insights into the potential and constraints of utilising LLMs for steganographic applications, but it has also highlighted areas that require more investigation. The challenges associated with information restoration, computational load, and detector performance highlight how difficult it is to hide and find concealed information in textual data. Even with these developments, creative solutions are still needed to keep up with the rapid breakthroughs in steganography and text analysis technology.

References

[Aga22] Neeraj Agarwal. N-gram language modeling in natural language processing. *KD nuggets*, 8, 2022.

[BAS+22] A Stevie Bergman, Gavin Abercrombie, Shannon Spruit, Dirk Hovy, Emily Dinan, Y-Lan Boureau, Verena Rieser, et al. Guiding the release of safer e2e conversational ai through value sensitive design. In *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*. Association for Computational Linguistics, 2022.

[Che21] Kinder Chen. Introduction to natural language processing — tf-idf. *Medium*, 8, 2021.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.

[EMM20] Oleg Evsutin, Anna Melman, and Roman Meshcheryakov. Digital steganography and watermarking for digital images: A review of current research directions. *IEEE Access*, 8:166589–166611, 2020.

[HBM+22] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[HXZ+22] Xuanli He, Qiongkai Xu, Yi Zeng, Lingjuan Lyu, Fangzhao Wu, Jiwei Li, and Ruoxi Jia. Cater: Intellectual property protection on text generation apis via conditional watermarks, 2022.

[KGW+23a] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. A watermark for large language models, 2023.

[KGW+23b] John Kirchenbauer, Jonas Geiping, Yuxin Wen, Manli Shu, Khalid Saifullah, Kezhi Kong, Kasun Fernando, Aniruddha Saha, Micah Goldblum, and Tom Goldstein. On the reliability of watermarks for large language models, 2023.

[MDK+23] Yisroel Mirsky, Ambra Demontis, Jaidip Kotak, Ram Shankar, Deng Gelei, Liu Yang, Xiangyu Zhang, Maura Pintor, Wenke Lee, Yuval Elovici, et al. The threat of offensive ai to organizations. *Computers & Security*, 124:103006, 2023.

[TLI+23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[VSP+23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.

[ZZL+23] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Z. Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jianyun Nie, and Ji rong Wen. A survey of large language models. *ArXiv*, abs/2303.18223, 2023.